

```

1  '''gridlocalize.py
2
3      This is the skeleton code for the grid-based localization.
4
5      PLEASE FINISH WRITING THE CODE, especially where marked by FIXME.
6
7  '''
8
9  import numpy as np
10
11  from gridutilities import Visualization, Robot
12
13
14  #
15  # Define the Walls
16  #
17  w = ['xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx',
18       'x              xx              xx              x',
19       'x              xx              xx              x',
20       'x              xx              xx              x',
21       'x      xxxxx      xx      xx              x',
22       'x      x  xx      xx      xx              x',
23       'x      x  xx      xx  xx      xxxxx      x',
24       'x      x   xx      xx xx      xxx   xxx      x',
25       'x      x   xx      xxx      xx      xx      x',
26       'x      x   xx      x      x      x      x      x',
27       'x      x   xx      xx      xx      xx      x',
28       'x      x      x      x      x      x      x',
29       'x      x      x      x      x      x      x',
30       'x      x      x      x      x      x      x',
31       'x              xx      xx      x      x',
32       'x              x      x              x',
33       'x              xx      xx              x',
34       'x              xxx   xxx              x',
35       'x              xxxxx              x      x',
36       'x              x              x      xx',
37       'x              x              x      xxx',
38       'x              x              x      xxxxx',
39       'x              xxx              x      xxxxx',
40       'x      xxxxx              x      xxxxxx',
41       'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx']
42
43  walls = np.array([[1.0*(c == 'x') for c in s] for s in w])
44  rows = np.size(walls, axis=0)

```

```

45 cols = np.size(walls, axis=1)
46
47
48 #####
49 #
50 # PREDICTION
51 #
52 # Input:  bel          Grid of probabilities (current belief)
53 #         drow, dcol   Commanded movement delta in row/col
54 #
55 # Output: prd          Grid of probabilities (prediction)
56 #
57 def computePrediction(bel, drow, dcol):
58     # Prepare an empty prediction grid.
59     prd = np.zeros((rows,cols))
60     free_cells = 0
61
62     # Determine the new probabilities (remember to consider walls).
63     # FIXME: Build up the grid of probabilities...
64
65     # Loop through all of the rows and columns
66     for row in range(0, rows):
67         for col in range(0, cols):
68             # Calculate the new position
69             new_row = row + drow
70             new_col = col + dcol
71
72             # Check if new row and new col are within bounds and isn't a wall
73             if (rows > new_row >= 0 and cols > new_col >= 0) and ((walls[new_row, new_col])
74                 prd[new_row, new_col] += 0.8 * bel[row, col]
75                 prd[row, col] += 0.2 * bel[row, col]
76             else:
77                 # Assign and add probabiltiy
78                 prd[row, col] += bel[row, col]
79
80     # Loop through all cells to find the free cells
81     for row in range(0, rows):
82         for col in range(0, cols):
83             if walls[row, col] == 0:
84                 # Keep count of number of cells that aren't walls
85                 free_cells += 1
86
87     uniform = 0.01 / free_cells
88
89     # Loop through cells, add uniform to free cells
90     for row in range(0, rows):

```

```

91         for col in range(0, cols):
92             if walls[row, col] == 0:
93                 prd[row, col] = 0.99 * prd[row, col] + uniform
94             else:
95                 prd[row, col] = 0.99 * prd[row, col]
96
97         # Check the prediction.
98         if abs(np.sum(prd) - 1.0) > 1e-12:
99             print("WARNING: Prediction does not add up to 100%")
100
101         # Return the prediction grid.
102         return prd
103
104     #
105     # MEASUREMENT UPDATE (CORRECTION)
106     #
107     # Input:  prior           Grid of prior probabilities (belief)
108     #         probSensor      Grid of modeled probabilities that (sensor==True)
109     #         sensor          Actual value of sensor (True/False)
110     #
111     # Output: post           Grid of posterior probabilities (updated belief)
112     #
113     def updateBelief(prior, probSensor, sensor):
114         post = np.zeros((rows, cols))
115         total = 0
116
117         # Create the posterior belief.
118         for row in range(0, rows):
119             for col in range(0, cols):
120                 if sensor:
121                     post[row, col] = prior[row, col] * probSensor[row, col]
122                 else:
123                     post[row, col] = prior[row, col] * (1 - probSensor[row, col])
124             total += post[row, col]
125
126         # Re-normalize so total sum of probabilities adds up to 1
127         for row in range(0, rows):
128             for col in range(0, cols):
129                 post[row, col] = post[row, col] / total
130
131         # Check the updated belief.
132         if abs(np.sum(post) - 1.0) > 1e-12:
133             print("WARNING: Belief does not add up to 100%")
134
135         # Return the updated belief.
136         return post

```

```

137
138
139 #####
140 #
141 # Pre-compute the Modeled Sensor Probability Grid
142 #
143 # Input: drow, dcol Sensor direction in row/col
144 #
145 # Output: prob Grid of modeled probabilities that (sensor==True)
146 #
147 def precomputeSensorProbability(drow, dcol):
148     # Prepare an empty probability grid.
149     prob = np.zeros((rows, cols))
150     probSensor = [0.9, 0.6, 0.3]
151
152     # Pre-compute the sensor probability on the grid.
153     for row in range(0, rows):
154         for col in range(0, cols):
155             p = 0.0
156
157             for d in range(1, 4):
158                 new_row = row + d * drow
159                 new_col = col + d * dcol
160
161                 # If new cell is out of bounds or there's a wall at that distance
162                 if (not (rows > new_row >= 0 and cols > new_col >= 0)) or (walls[new_row, new_col]):
163                     p = probSensor[d - 1]
164                     break
165
166             prob[row, col] = p
167
168     # Return the computed grid.
169     return prob
170
171 #####
172 #
173 # Main Code
174 #
175 def main():
176     # Initialize the robot simulation.
177     # FIXME... PICK THE "REALITY" WE SHOULD SIMULATE:
178
179     # robot=Robot(walls)
180     # 4a
181     # robot=Robot(walls)
182

```

```

183     # 4b
184     # robot=Robot(walls, row=12, col=26)
185     # 5
186     # robot=Robot(walls, row=12, col=26, pSensor=[0.9,0.6,0.3])
187     # 6
188     # robot=Robot(walls, row=15, col=47, pSensor=[0.9,0.6,0.3], pCommand=0.8)
189     # 7
190     robot=Robot(walls, row= 7, col=12, pSensor=[0.9,0.6,0.3], pCommand=0.8, kidnap=True)
191     # Or to play:
192     #     robot=Robot(walls, pSensor=[0.9,0.6,0.3], pCommand=0.8)
193
194
195     # Initialize the figure.
196     visual = Visualization(walls, robot)
197
198
199     # Pre-compute the probability grids for each sensor reading.
200     probUp    = precomputeSensorProbability(-1, 0)
201     probRight = precomputeSensorProbability( 0, 1)
202     probDown  = precomputeSensorProbability( 1, 0)
203     probLeft  = precomputeSensorProbability( 0, -1)
204
205     # Show the sensor probability maps.
206     visual.show(probUp,    "Probability of up    proximal sensor reading True")
207     visual.show(probRight, "Probability of right proximal sensor reading True")
208     visual.show(probDown,  "Probability of down  proximal sensor reading True")
209     visual.show(probLeft,  "Probability of left  proximal sensor reading True")
210
211
212     # Start with a uniform belief grid.
213     bel = (1.0 - walls) / np.sum(1.0 - walls)
214
215     # Loop continually.
216     while True:
217         # Show the current belief. Also show the actual position.
218         visual.show(bel, markRobot=True)
219
220         # Get the command key to determine the direction.
221         while True:
222             key = input("Cmd (q=quit, i=up, m=down, j=left, k=right) ?")
223             if (key == 'q'): return
224             elif (key == 'i'): (drow, dcol) = (-1, 0) ; break # up
225             elif (key == 'k'): (drow, dcol) = ( 0, 1) ; break # right
226             elif (key == 'm'): (drow, dcol) = ( 1, 0) ; break # down
227             elif (key == 'j'): (drow, dcol) = ( 0, -1) ; break # left
228

```

```

229         # Move the robot in the simulation.
230         robot.command(drow, dcol)
231
232
233         # Compute a prediction.
234         prd = computePrediction(bel, drow, dcol)
235         #visual.show(prd, "Showing the prediction")
236
237         # Correct the prediction/execute the measurement update.
238         bel = prd
239         bel = updateBelief(bel, probUp,    robot.sensor(-1, 0))
240         bel = updateBelief(bel, probRight, robot.sensor( 0, 1))
241         bel = updateBelief(bel, probDown,  robot.sensor( 1, 0))
242         bel = updateBelief(bel, probLeft,  robot.sensor( 0, -1))
243
244
245     if __name__ == "__main__":
246         main()

```